Assignment 3: Experimental Engineering

Performance, mutation, and chaos testing

Spring 2026
25 April 2026

SUBMITTED TO

Astana IT University
Faculty of Engineering
Software QA and Testing
AQA
Course Instructor

SUBMITTED BY

Aldiyar Seylkhanov, Alexandr Tyulkov, Malika Ishakhanova

Contents

1. Introduction

This report evaluates whether the current Circles backend test setup is ready for experimental engineering work. The focus is on three activities: performance testing, mutation testing, and a simple chaos experiment. The goal is not to prove production readiness. The goal is to measure what the current codebase can already support and to identify the main gaps.

The work was performed on 25 April 2026 in the local monorepo. Only commands already supported by the repository were used. Dependency installation, checks, and tests were executed through the `vp` toolchain. Runtime experiments were executed against the backend application in development mode.

2. Experimental Setup

2.1. Environment

Item	Value	Notes
Workspace root	`circles`	Monorepo managed through Vite+
Package tool	`vp v0.1.18`	Local `vite-plus` version `0.1.15`
Node.js	`v24.15.0`	Matches the current workspace runtime
Backend test runner	`vp test`	Vitest through Vite+
Mutation tool	`stryker run`	Configured in `apps/backend/stryker.config.json`
Load tool	`vp dlx autocannon`	Used for repeatable local HTTP benchmarks
Database	PostgreSQL 16	Dedicated container bound to port `5433`
Dataset	Synthetic seed	7 users, 30 tracks, 16,600 history rows

Table 1: Execution environment used for the experiments

The default local PostgreSQL port `5432` was already occupied by another service on the machine. For that reason, the runtime experiments used a separate PostgreSQL 16 container on port `5433` and temporary environment overrides such as `DATABASE_URL=postgres://circles:password@127.0.0.1:5433/circles_dev`. This choice kept the repository files unchanged and made the experiment reproducible.

The authenticated benchmark used the built-in endpoint `POST /test/e2e/login` with `ENABLE_E2E_AUTH=true`. This endpoint generated a valid session cookie without depending on Spotify OAuth. A small synthetic seed was then inserted into the

migrated schema so that the `leaderboard` and `library overview` queries would read real data instead of empty tables.

2.2. Executed Commands

Command	Result	Purpose
`vp install`	Passed	Verified workspace dependencies and lockfile state
`vp check`	Passed	Confirmed formatting, linting, and type checks
`vp run @circles/backend#test`	Passed	Executed backend unit tests
`vp run @circles/backend#test:coverage`	Failed threshold	Measured backend coverage against configured global thresholds
`vp run @circles/backend#test:mutation`	Passed	Measured mutation score for selected library files
`vp run @circles/backend#pg:migrate`	Passed	Applied the current database schema to the isolated PostgreSQL instance
`vp dlx autocannon -c 20 -d 10 ...`	Passed	Measured endpoint latency and throughput under concurrent local load
`docker stop/start circles-postgres-aqa`	Passed	Injected and recovered a database outage

Table 2: Commands used during the assignment run

3. Baseline Validation

The repository was ready for execution without any code changes before the experiments began. `vp install` completed successfully and reported that the lockfile was already up to date. `vp check` then passed, with formatting, linting, and type checks all reported as clean. This result is important because later failures can be attributed to experiment conditions rather than obvious repository breakage.

The backend unit suite also passed without modification. The command `vp run @circles/backend#test` executed 7 test files and 33 tests, and all of them passed. The tests mainly target small library helpers such as range conversion, retry logic, ZIP detection, and Spotify export parsing. This is a useful base, but it is still a narrow slice of the service.

Coverage exposed that limitation immediately. The command `vp run @circles/backend#test:coverage` still executed the same 33 tests successfully, but the global thresholds failed. Measured coverage was 5.88% for statements, 6.83% for branches, 5.45% for functions, and 5.17% for lines. The configured thresholds are 90% for lines, statements, and functions, and 85% for branches. The current suite is therefore good enough for targeted helper validation, but not strong enough to act as a whole-backend quality gate.

Check	Outcome	Evidence
`vp install`	Pass	Workspace dependencies were already current
`vp check`	Pass	183 files formatted; 147 files clean for lint and type checks
`vp run @circles/backend#test`	Pass	7 test files passed; 33 tests passed
`vp run @circles/backend#test:coverage`	Threshold fail	All tests passed, but overall backend coverage remained below the configured global gate

Table 3: Baseline validation summary

4. Performance Testing

4.1. Method

Performance testing was executed against a live backend server started with `vp run @circles/backend#dev:api` on port `8001`. The isolated PostgreSQL instance on port `5433` was used for all database-backed requests. Each benchmark used `autocannon` with 20 concurrent connections and a 10 second duration. Raw outputs were saved as JSON and then reduced to latency and throughput metrics.

Three endpoints were selected:

- `/health` as a lightweight public baseline.
- `/leaderboard?period=all` as a cached aggregate query.
- `/library/overview?range=30d` as an authenticated user-specific query.

During setup, an additional behavior was discovered: `/leaderboard?period=all` returned `401 Unauthorized` when called without a session cookie, even though the controller itself does not define an auth middleware. The benchmark therefore used the E2E session cookie. This should be treated as an observed routing or middleware defect in the current backend assembly.

4.2. Measured Results

Endpoint	Auth	Avg Latency	P99	Avg RPS	Notes
`/health`	No	0.01 ms	0 ms	43,093.82	Very small handler with no database access
`/leaderboard?period=all`	Yes	13.83 ms	24 ms	1,396.70	Aggregated read with cached result path
`/library/overview?range=30d`	Yes	12.62 ms	21 ms	1,523.90	Authenticated database-backed summary query

Table 4: Local load-test results with 20 concurrent connections over 10 seconds

The measured latencies were comfortably below the informal coursework target of 200 ms for read requests. Throughput also exceeded 100 requests per second by a large margin. However, these numbers should be interpreted carefully. The experiment used a local machine, a small synthetic dataset, and a development server. The results show that the current implementation is lightweight under local conditions. They do not prove behavior under production traffic, remote network latency, or larger datasets.

The most meaningful endpoint is `/library/overview?range=30d`, because it requires authentication and performs multiple database reads. Its average latency remained close to 13 ms and its P99 remained below 25 ms on the seeded dataset. This suggests that the current query structure is acceptable for coursework-scale use, at least before real-world network and infrastructure costs are added.

The `/health` result is intentionally extreme because the handler only returns a short JSON body and does not touch external dependencies. It is useful as a lower bound, but not as a business-level performance indicator.

5. Mutation Testing

5.1. Method

Mutation testing used the repository configuration in `apps/backend/stryker.config.json`. Only four helper modules were included in the mutation set:

- `src/library/range.ts`
- `src/library/retry.ts`

- `src/library/spotify-export.ts`
- `src/library/zip.ts`

This narrow scope is important. The mutation score says a great deal about those helper tests, but very little about the API, authentication, workflows, or persistence layer.

5.2. Measured Results

Stryker generated 58 mutants and finished in 29 seconds. The final mutation score was 87.93%. Fifty mutants were killed, seven survived, and one timed out. The score is above the default high threshold of 80%, so the selected helper tests are reasonably strong.

File	Total	Killed	Survived	Timeout	Score
`range.ts`	8	8	0	0	100.00%
`retry.ts`	26	19	6	1	76.92%
`spotify-export.ts`	16	16	0	0	100.00%
`zip.ts`	8	7	1	0	87.50%
All selected files	58	50	7	1	87.93%

Table 5: Mutation testing summary

The weakest area was `retry.ts`. Surviving mutants showed that the current tests do not fully constrain retry loop boundaries, delay calculations, or logging side effects. One surviving mutant also remained in `zip.ts`, where the test suite did not fully reject a weakened signature check. These results are still useful because they identify specific places where a small number of targeted test additions could improve fault detection.

The strongest result came from `range.ts` and `spotify-export.ts`, both of which achieved 100%. For these two helpers, the tests currently express the intended behavior in a strict and stable way.

6. Chaos Testing

6.1. Method

The chaos experiment targeted the most important local dependency: PostgreSQL. The backend server stayed running while the database container `circles-postgres-aqa` was stopped and started. The observed request path was the authenticated endpoint `/library/overview?range=30d`, because it requires both session lookup and application data access.

The following sequence was executed:

1. Start the backend server against PostgreSQL on port `5433`.
2. Create a valid session by calling `POST /test/e2e/login`.

3. Confirm that `/library/overview?range=30d` returns `200`.
4. Stop the PostgreSQL container.
5. Call the same endpoint again and record the result.
6. Start PostgreSQL again, wait briefly, and call the endpoint once more.

6.2. Measured Results

Stage	HTTP	Observed Behavior
Before failure	200	Authenticated library overview returned valid JSON with seeded totals
Database stopped	500	Endpoint failed with `{\"status\":500, \"message\":\"Failed to get session\"}`
Error source	n/a	Backend logs showed `ECONNREFUSED 127.0.0.1:5433` during Better Auth session lookup
Database restarted	200	The same endpoint recovered after the database returned; no API restart was needed

Table 6: Chaos experiment against PostgreSQL availability

This result is mixed. The backend did fail immediately when the database became unavailable, which is expected for a service that stores sessions and application data in PostgreSQL. However, the service also recovered automatically once the database came back. That is a better outcome than a full process crash, but it still means user-visible errors are produced during the outage window.

The failure point also matters. The request did not fail deep inside the library query itself. It failed earlier during session retrieval in Better Auth. This means every authenticated route is exposed to the same availability risk, not only the library endpoint.

7. Discussion

The experiments show that the repository is ready for a limited but real experimental workflow. The automated scripts can be executed, the mutation tooling is working, and the backend can be stress-tested locally without inventing extra infrastructure. This is a strong improvement over a purely theoretical assignment.

At the same time, the experiments exposed three important gaps.

- Backend coverage is far too low to justify the current global coverage thresholds.
- Mutation testing only covers four helper files, so the high score cannot be generalized to the full service.
- The public `leaderboard` route currently behaves like an authenticated route during real execution, which indicates a routing or middleware composition problem.

These findings point to a clear next iteration. The highest-value improvement is not more benchmarking. It is broader test coverage around API controllers, authentication boundaries, and database-backed queries. Once those tests exist, the same performance and chaos procedures can be rerun on a more representative service surface.

8. Conclusion

The current Circles backend is ready for basic experimental engineering work, but only within a narrow scope. The automated baseline is stable, the selected helper tests have a strong mutation score of 87.93%, and the local performance results are fast on a coursework-scale dataset. The service also recovers after a short PostgreSQL outage without a manual restart.

The main limitation is representativeness. Coverage is still below 7% across the backend, mutation testing touches only helper modules, and a real routing issue was found while preparing the performance run. The project is therefore in a good position for further experiments, but not yet in a position where those experiments should be treated as full-system evidence.